Tema 4

Elaboración de diagrama de clases

Entornos de Desarrollo

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Notación de los diagramas de clases 5
2.1 Clase 5
2.2 Clases y métodos abstractos 6
2.3 Interfaces 7
2.4 Relaciones 7
2.4.1 Relaciones de generalización o herencia 7
2.4.2 Relación de realización o de implementación 8
2.4.3 Relación de asociación 9
2.4.4 Relación de dependencia o de uso 10
2.4.5 Relación de composición 10
2.4.6 Relación de agregación 11
2.4.7 Resumen de las relaciones 12
3. Utilización de herramientas CASE para elaborar diagrama de clases 13
4. Módulos integrados en entornos de desarrollo para elaborar diagramas de clases 16
5. Creación de código a partir de diagramas de clases 17
6. Generación de diagramas de clases a partir de código (ingeniería inversa) 20
7. Resumen 22
Glosario 23
Bibliografía 24

Objetivos de la unidad

- Conocer la **notación de UML** para la realización del diagrama de clases, incluidos elementos básicos como atributos y métodos, así como los distintos tipos de relaciones.
- Presentar algunas de las herramientas disponibles para la realización de diagramas de clases ya sean de escritorio, a través de la web o mediante componentes añadidos a los IDE de desarrollo.
- Descubrir la utilidad de la **generación automática de código** basado en el diagrama de clases.
- Aplicar la **ingeniería inversa** para obtener el diagrama de clases a partir de código fuente existente.

Tech Hub <FP> euroformac

1. Introducción

Teniendo en cuenta que **UML** permite representar de manera formal diferentes aspectos y perspectivas de un sistema orientado a objetos, el **diagrama de clases** es, probablemente, el más interesante e importante de los diagramas de que contempla este lenguaje.

En el diagrama de clases se reflejan, como su propio nombre indica, las clases que componen un sistema *software*, incluyendo sus atributos, métodos y relaciones. Este diagrama forma parte del diseño del *software* y permite a los desarrolladores escribir los programas a partir de una especificación formal.

La construcción de los diagramas de clase se podría realizar de manera artesanal utilizando un papel y un bolígrafo, pero no sería propio de un entorno profesional dedicado al *software*. Existen herramientas capaces de dar soporte a la creación de los diagramas orientados a objetos, incluyendo habitualmente generadores de código, lo que permite automatizar parte del trabajo de programación. Del mismo modo, estos programas pueden incluir herramientas para obtener los diagramas a partir del código fuente, lo que se conoce como ingeniería inversa.

2. Notación de los diagramas de clases

El **diagrama de clases UML** se construye a partir de una notación precisa capaz de reflejar todas las características de las clases.

En este apartado se muestran los elementos gráficos más importantes de los diagramas de clase.

2.1 Clase

Se representa mediante un rectángulo dividido verticalmente en tres áreas que contienen el **nombre de la clase** (área superior), **atributos** (área intermedia) y **métodos** (área inferior).

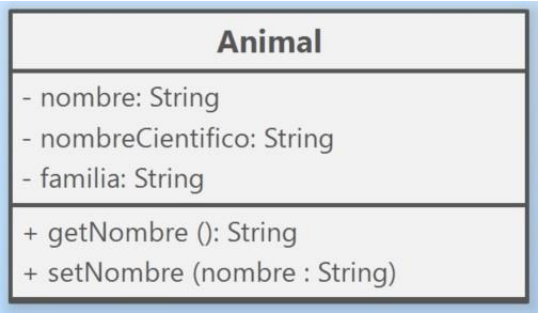
Los atributos y métodos van precedidos del modificador de acceso correspondiente que se asigna según los siguientes símbolos:

Símbolo	Visibilidad
+	Público
-	Privado
#	Protegido
~	De paquete

En la sección correspondiente a los atributos, después del modificador de acceso, se indica el nombre de estos seguido del signo **dos puntos** (**:**) y del tipo del atributo.

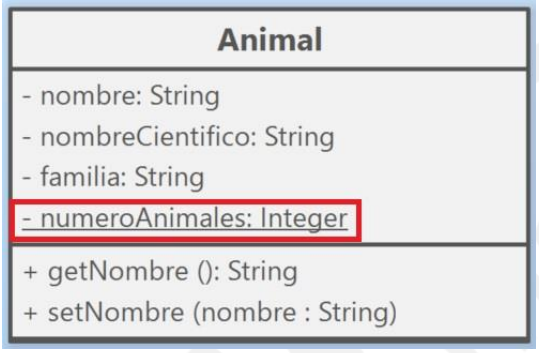
Por su parte, los métodos se definen con el modificar de acceso, seguido del nombre del método, la lista de atributos separados por comas y dentro de paréntesis, el signo de dos puntos y el tipo de retorno. Los parámetros y sus tipos van separados por el signo de dos puntos. Los constructores, por su parte, son representados como un método más.

En la siguiente imagen se muestra el diseño UML de la clase "Animal", compuesta por los atributos "nombre", "nombreCientifico" y "familia", todos definidos como privados y de tipo ***String***. Además, la clase contiene los métodos públicos "getNombre" que no recibe parámetros y devuelve un valor de tipo *String* y "setNombre", también público, que recibe el parámetro "nombre" de tipo *String* y no tiene retorno.



Clase UML

Para indicar que un atributo es estático, este debe ir subrayado.

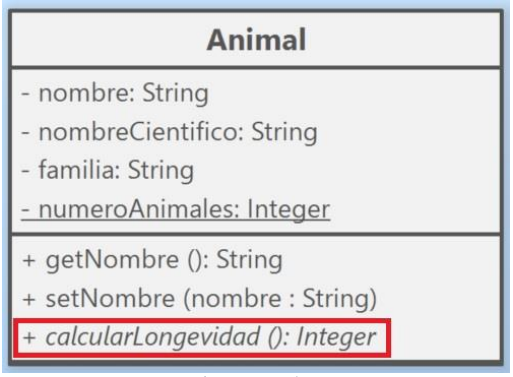


Atributo estático

Las constantes (atributos declarados como *finales*) deberán escribirse en mayúsculas separando cada palabra por un guion bajo, como por ejemplo ATRIBUTO_FINAL.

2.2 Clases y métodos abstractos

Los nombres de las clases y los métodos abstractos se escriben en cursiva, diferenciándose así de los que no lo son.



Método estático

2.3 Interfaces

Las interfaces tienen la misma estructura que las clases, diferenciándose en que incorporan encima del nombre de la clase la palabra ***interface*** entrecomillada y en letra cursiva.

En la siguiente imagen se muestra el diseño de la interface "IMovimientos", compuesta por los métodos público "avanzar", "retroceder" y "saltar".

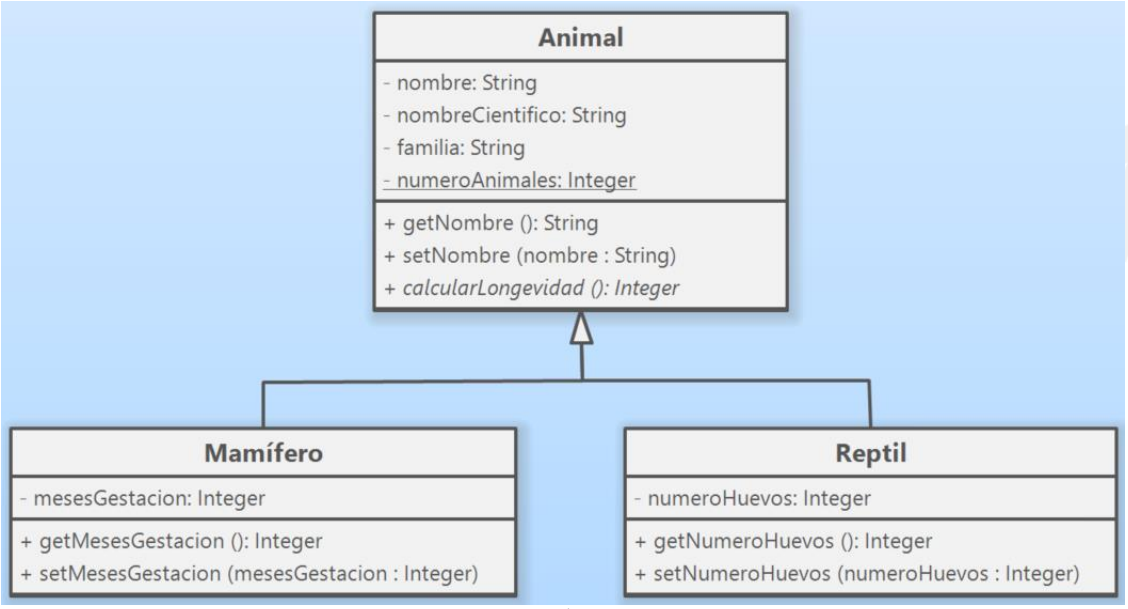
Interfaz

2.4 Relaciones

UML proporciona la notación suficiente para representar todas las relaciones que se pueden dar entre las clases de un diagrama.

2.4.1 Relaciones de generalización o herencia

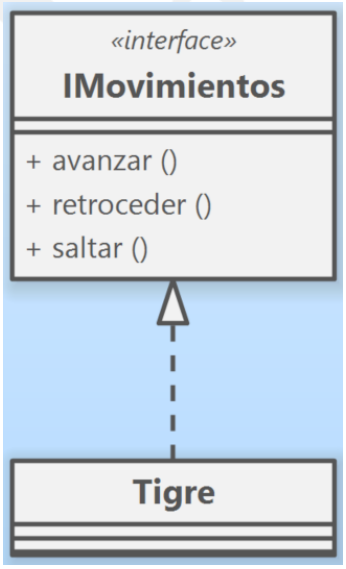
La **herencia** en UML se denomina **generalización**. En la relación de generalización una clase (hija o derivada) es subclase de otra (base o superclase). En el diagrama de clases se representa la relación con una línea que une la clase derivada con la clase base rematada con una punta de flecha hueca (solo el contorno, o rellena de color blanco).



Generalización o herencia

2.4.2 Relación de realización o de implementación

Cuando una clase implementa una interfaz (conocido en UML como **realización**), la relación se refleja mediante una línea punteada que termina en una punta de flecha hueca (solo el contorno, o rellena de color blanco) que va desde la clase hacia la interfaz.



Realización o implementación de interfaz

2.4.3 Relación de asociación

La relación de asociación en UML representa los vínculos que existen entre clases que no están relacionadas jerárquicamente. Sin la relación de asociación, los diagramas de clases estarían compuestos por clases aisladas, sin vínculos.

En UML las relaciones de asociación se representan mediante una línea sólida que conecta las clases afectadas. En los extremos de las líneas se puede indicar la **multiplicidad** (cuántas instancias de una clase puede haber en la relación): 1 (uno), * (cero o varios), 0.1 (uno opcional), 0.* (ninguno o varios), 1.* (uno o varios), así como la **navegabilidad** (si desde uno de los objetos de la relación objeto se puede obtener una referencia al otro) mediante una punta de flecha .

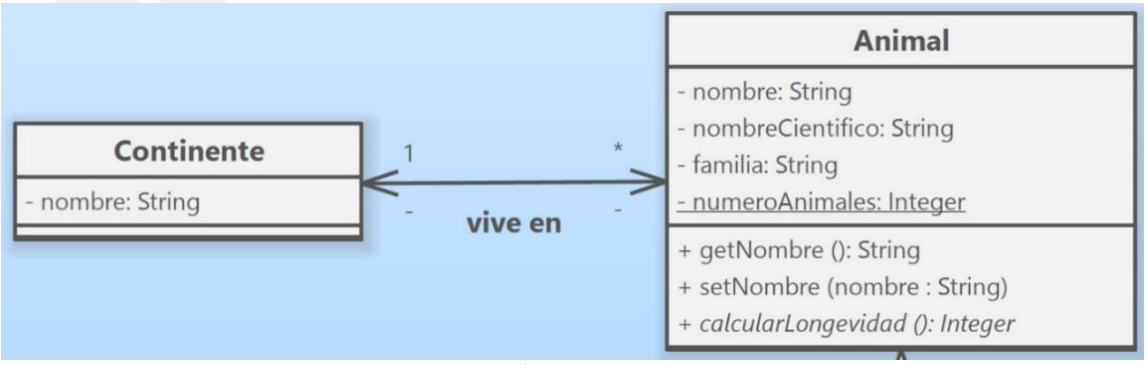
Símbolo de navegabilidad

Si, por el contrario, la navegación no es posible, el símbolo del extremo de la línea al que no se puede navegar es una equis (X).

Símbolo de no navegabilidad

Las clases también pueden ir identificadas por un nombre en el que se indica el tipo de relación que tienen entre sí.

En la siguiente figura se muestra la representación de una relación de asociación entre las clases "Animal" y "Continente" con el nombre "vive en". Según la cual, un objeto de la clase "Animal" tiene una relación de asociación con un objeto de la clase "Continente" y, por otra parte, un objeto de clase "Continente" tiene una relación de asociación con varios objetos de la clase "Animal".



Relación de asociación

2.4.4 Relación de dependencia o de uso

Esta relación representa el uso que hace una clase de, normalmente, la funcionalidad proporcionada por otra.

Se representa mediante una línea discontinua remada en una punta de flecha que apunta a la clase que proporciona la funcionalidad. El tipo de uso se puede especificar mediante un texto entrecomillado, siendo *"use"* el correspondiente a la relación puramente de uso.

En la siguiente imagen se muestra la relación de uso entre una clase encargada de enviar correos electrónicos (GestorEmail) y otra capaz de generar documentos PDF a partir de unos datos proporcionados (GeneradorPDF). El diagrama podría representar que el gestor de correos utiliza el generador de documentos PDF para crear los documentos en este formato para enviarlo al destinatario.

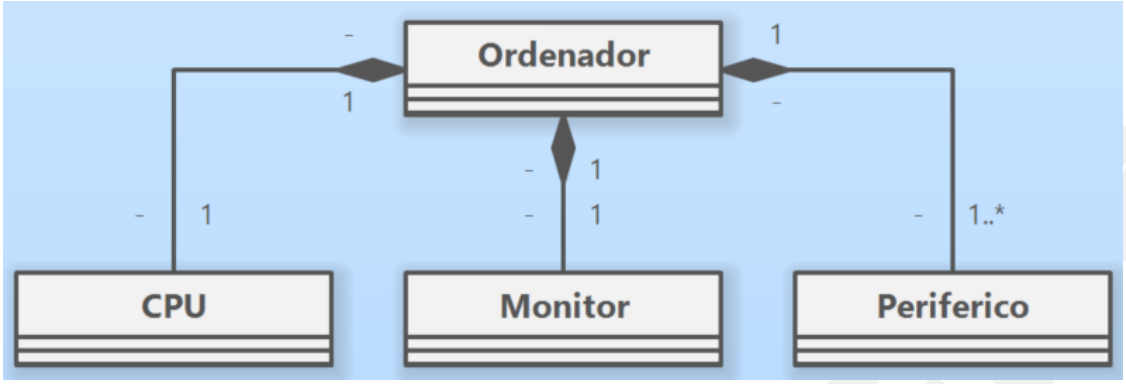
Relación de uso

2.4.5 Relación de composición

La composición es un tipo de relación de asociación en la que una clase está compuesta por otras (**composición fuerte**) y que todas ellas tienen sentido en conjunto y no de manera independiente.

Se representa con una línea sólida rematada con el símbolo de un diamante sólido en el extremo de la clase que representa el conjunto. Los extremos de la relación pueden contener información sobre la multiplicidad.

En la siguiente imagen se representa una relación de composición en la que se indica que la clase que representa un ordenador está compuesta por una CPU, un monitor y uno o más periféricos. En este tipo de relaciones, si la composición se elimina, se eliminan las partes, por lo que las partes (CPU, monitor y periféricos) no pueden existir si no existe el ordenador.



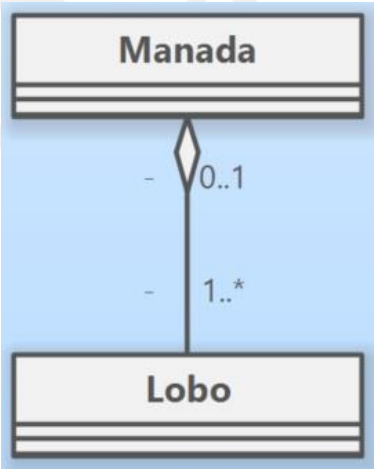
Relación de composición

2.4.6 Relación de agregación

La agregación es un tipo de relación de asociación en la que una clase es parte de otra (**composición débil**). De alguna manera, las partes se unen para construir una entidad más completa. La eliminación de la composición no elimina todas las partes, ya que las agregadas tenían entidad propia.

Se representa con una línea sólida rematada con el símbolo de un diamante hueco en el extremo de la clase que representa el conjunto. También pueden incluir información sobre la multiplicidad.

La siguiente imagen muestra una relación de agregación en la que se presenta que una manada está compuesta por varios lobos. Si la relación se elimina, los lobos seguirán teniendo entidad.



Relación de agregación

2.4.7 Resumen de las relaciones

A modo de resumen, la siguiente tabla recoge los nombres de las relaciones y los símbolos de representación utilizados en las mismas.

Relación	Representación
Generalización (Herencia)	
Realización (Implementación)	
Asociación	
Dependencia	
Composición	
Agregación	

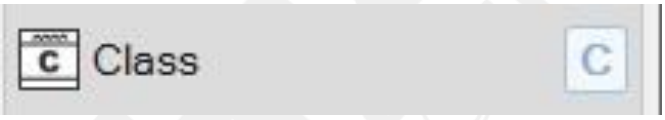
3. Utilización de herramientas CASE para elaborar diagrama de clases

Las herramientas CASE proporcionan utilidades para construir diagramas de clases. En concreto, la aplicación *Software Ideas Modeler*, dispone de un editor potente y sencillo que ofrece toda la funcionalidad necesaria para el diseño de este tipo de diagramas.

A continuación, se muestran los pasos para construir un diagrama de clases compuesto que cumpla con las siguientes especificaciones:

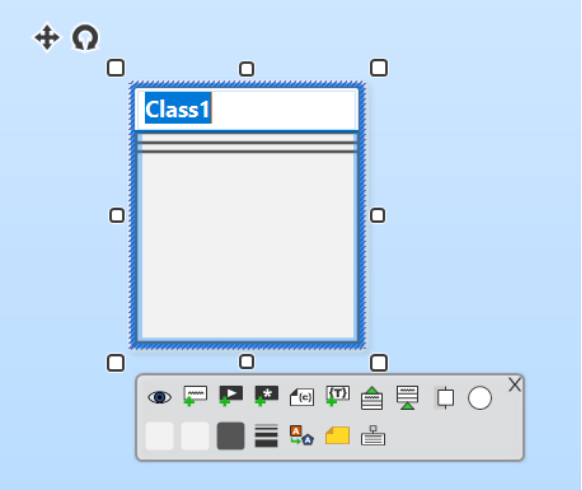
- Una clase base *Usuario* compuesta por el atributo *nombre* de tipo *String*, un constructor y el método *getNombre*.
- Tres clases hija o derivadas de *Usuario* con los nombres *Operador*, *Administrador* y *Responsable*.

Una vez creado el proyecto y elegido el diagrama de clases, para insertar clases se debe pulsar sobre la opción "Class" del menú izquierdo.



Opción de creación de clase

Una vez seleccionada la opción "Class" se debe pulsar sobre el lienzo del diagrama. Se muestra entonces una clase vacía con un nombre asignado por defecto que deberá cambiarse pulsando sobre el mismo y escribiendo el nombre real de la clase.

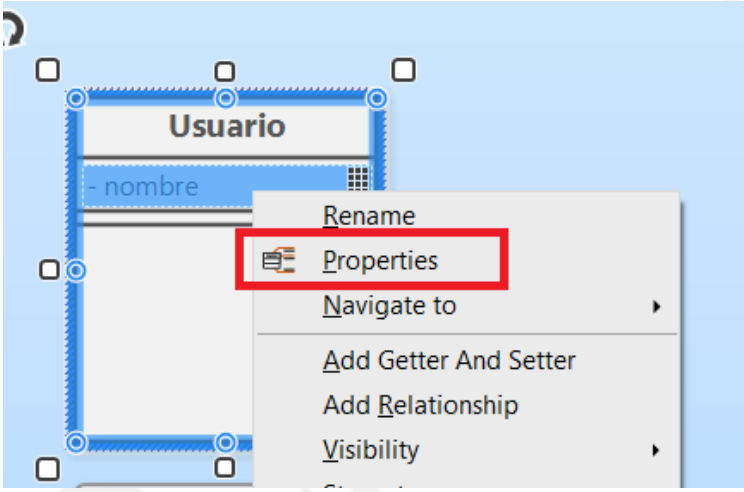


Creación de clase

En el menú flotante que aparece debajo de la clase, se encuentran los iconos para acceder a la configuración de la clase. Remarcados en rojo y de izquierda a derecha, se encuentran los botones para agregar atributos, operaciones (métodos) y constructores.

Botones de creación de atributos, operaciones (métodos) y constructores

Una vez agregados los atributos, métodos y/o constructores, pulsando con el botón derecho sobre los mismos se puede acceder al menú de propiedades, desde donde se podrán indicar los tipos de datos de los atributos, así como los nombres y tipos de los parámetros de los métodos y de los constructores.

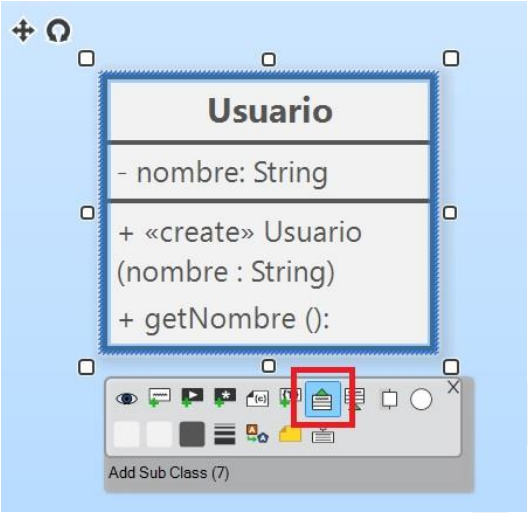


Acceso a las propiedades de un atributo

Sabías que

La herramienta CASE *Software Ideas Modeler* trata a los constructores de manera distinta a los métodos, añadiendo el prefijo <<*create*>> al nombre de aquellos para una mejor identificación.

Una vez creados atributos, constructores y métodos, se puede agregar una subclase pulsando sobre el icono correspondiente.



Creación de relación de herencia

En este caso, dado que la clase tiene 3 subclases, pulsaremos tres veces sobre la acción correspondiente creando una jerarquía.

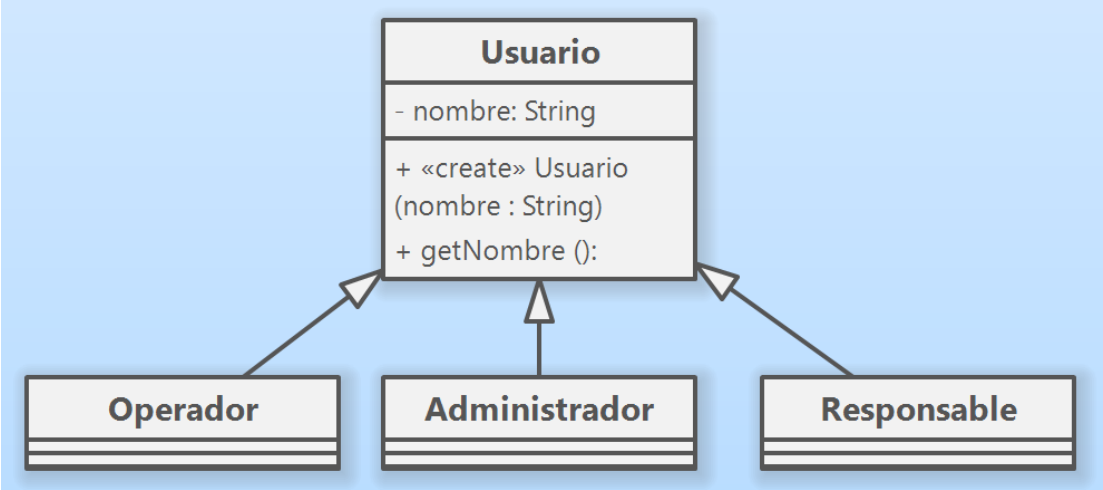


Diagrama de clases con herencia

Como se puede suponer, la funcionalidad proporcionada por las herramientas CASE es mucho más extensa, exigiendo el aprendizaje de la herramienta concreta que se utilice para poder llevar a cabo el trabajo de diseñar un diagrama de clases completo y ajustado a las necesidades del sistema software que se desee desarrollar.

4. Módulos integrados en entornos de desarrollo para elaborar diagramas de clases

Los entornos de desarrollo pueden incluir herramientas para la construcción de diagramas de clases.

El **IDE PyCharm**, por ejemplo, en su versión profesional dispone de un complemento opcional (*plugin*) llamado "*Diagrams*" capaz de representar el diagrama de clases de una aplicación Python.

Más info

Se puede obtener más información del *plugin* "*Diagrams*" en el siguiente enlace:

Ver enlace.

Por su parte, el **IDE NetBeans** dispone de un *plugin* similar conocido como "*PlantUML-NB*".

Más info

Se puede obtener más información del *plugin* "*PlantUML-NB*" en el siguiente enlace:

Ver enlace.

Otro IDE que puede extenderse para proporcionar la capacidad de diseñar diagramas de clase es **Eclipse**. Dispone de varios *plugins*, como "*UML Lab Modeling IDE*" o "*UMLet*", por citar solo algunos de varios disponibles.

Más info

Se puede obtener más información del *plugin* "*UML Lab Modeling IDE*" y "*UMLet*" en los siguientes enlaces:

Ver enlace 1.

Ver enlace 2.

5. Creación de código a partir de diagramas de clases

Una de las aportaciones más atractivas para los programadores, por parte de las herramientas de diseño de diagrama de clases, es el generador de código. Esta utilidad permite, a partir de un diagrama de clases, crear código fuente en el lenguaje deseado de manera automática.

Lógicamente, esta automatización no sustituye al trabajo del programador, ya que no implica la creación del sistema *software* completo, pero sí le evita un trabajo que, por lo general, es sencillo pero tedioso: definición de las clases e interfaces, atributos, constructores, declaración de los métodos y la relaciones entre clases (de herencia, asociación, etc.).

Por ejemplo, la herramienta *online* ***GenMyModel*** (https://www.genmymodel.com/) dispone, entre otras, de una herramienta de diseño de diagramas de clases con generación de código.

A partir de un diagrama de clases como el que se muestra en la siguiente figura, creado desde la propia aplicación, se puede generar el código fuente básico que contenga la información presentada en la clase.

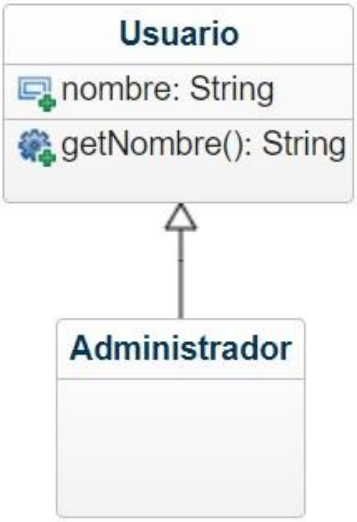
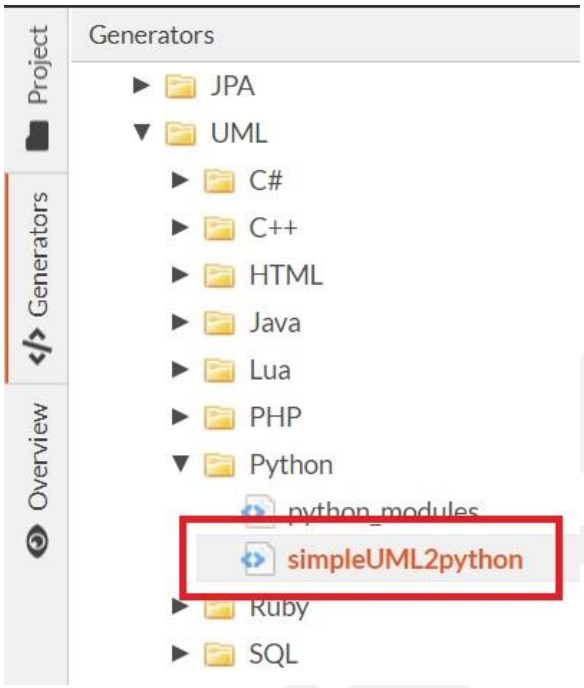


Diagrama de clases

En el área correspondiente a los generadores (pestaña *Generators*) de *GenMyModel* está disponible el menú *UML* en el que se presentan diferentes lenguajes y opciones para la generación del código fuente.



Selección de generador de código

Como ejemplo, utilizando el generador de código Python *simpletUML2python* con el diagrama de clases anterior, el sistema genera el siguiente código fuente (se han eliminado del mismo los comentarios para ahorrar espacio).

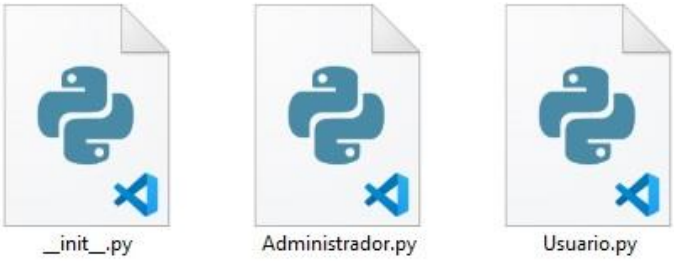
```
class Usuario(object):
    def __init__(self):
        self.nombre = ""

    def getNombre(self):
        return ""

class Administrador(Usuario):
    pass
```

Como se puede observar, el código contiene la definición de las clases (ambas en un único fichero), los atributos y la relación entre ambas.

De manera alternativa, eligiendo el generador *Python_modules* el resultado es diferente.

Selección de generador de código

En este caso, se generan tres ficheros: un fichero *main* o principal y otros dos correspondientes a las dos clases que tiene el diagrama.



Ficheros resultantes de la generación

Internamente, el código generado por ambos generadores es similar, aunque contiene pequeñas diferencias. En cualquier caso, estos ejemplos sirven para ilustrar cómo es posible generar código fuente a partir de los diagramas de clases en diferentes lenguajes o en el mismo lenguaje con diferentes técnicas.

6. Generación de diagramas de clases a partir de código (ingeniería inversa)

En general, hacer **ingeniería inversa** consiste en, a partir de un producto ya creado (físico o lógico), obtener información referente a cómo está diseñado y construido. Toma este nombre porque hace el camino inverso al realizado en la ingeniería tradicional, que comienza en la concepción y el diseño del producto y finaliza en su realización.

El concepto de ingeniería inversa también se aplica en la ingeniería del *software*. En concreto, en lo referente a los diagramas de clases, consiste en obtener dicho diagrama a partir del código fuente de una aplicación.

Para ilustrar este proceso de manera sencilla, se toma como entrada el siguiente código correspondiente a la declaración de una clase Python consistente en tres atributos, un constructor que recibe dos parámetros y dos métodos.

```
class Pelicula():
    def __init__(self, titulo, director):
        self.titulo = titulo
        self.director = director
        self.genero = ""

    def set_genero(self, genero):
        self.genero = genero

    def get_genero(self):
        return self.genero
```

La generación del diagrama de clases se va a realizar *GitUML*, una herramienta online que permite crear diagramas de clase a partir de repositorios GitHub o de ficheros subidos directamente. La versión gratuita está limitada, pero es muy sencilla de utilizar y es suficiente para mostrar el significado de la ingeniería inversa.

Accediendo a la web https://www.gituml.com/, abriendo el menú "*New Diagram*" y seleccionado la opción "*Upload Files...*", bastará con arrastrar el fichero con el código fuente Python y pulsar el botón "View UML" para obtener una imagen con el diagrama de clases contenido en el fichero.

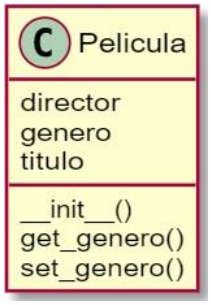


Diagrama generado con ingeniería inversa

Como se puede observar en la captura anterior, se ha creado la clase película, incluyendo los tres atributos, el constructor y los dos métodos.

Tech Hub <FP> euroformac

7. Resumen

El diagrama de clases es una herramienta fundamental para el diseño de aplicaciones de *software* orientadas a objetos. Gracias a UML se dispone de una notación estandarizada que permite que el diagrama refleje toda la riqueza de la programación orientada a objetos en relación con las clases.

Las herramientas **CASE** proporcionan la ayuda necesaria para poder trabajar con diagramas de clases con facilidad. Gracias a los módulos de diseño de diagramas, los generadores de código y los sistemas de ingeniería inversa, las herramientas CASE o los complementos disponibles para los IDEs, mejoran la eficiencia en el trabajo de diseño y codificación.

Tech Hub <FP> euroformac

Glosario

- **Abstracto.** Una clase es abstracta cuando al menos uno de sus métodos es abstracto. Un método es abstracto cuando no está programado.
- **Agregación.** Relación entre clases en la que una clase agrupa a otras.
- **Asociación.** Relación entre clases en la que una clase tiene como atributos instancias de otra.
- **Composición**. Relación entre clases en la que una clase está compuesta por otras.
- **Dependencia (uso).** Relación entre clases en la una clase utiliza funcionalidad de otra.
- **Generalización (herencia).** Relación entre clases en la que una de ellas (la clase hija o clase derivada) especializa a otra más general (clase base o superclase).
- **Interfaz**. En orientación a objetos, es una clase en la que únicamente se han declarado sus métodos, pero no se han programado.
- **Realización (implementación)**. Relación entre clases en la que una de ellas implementa una interfaz proporcionando una versión funcional de esta.
- **Visibilidad.** Aplicada a la orientación objetos, es la capacidad que tiene un objeto de ocultar o mostrar sus atributos y métodos a otros objetos.

Bibliografía

GenMyModel: online modeling made easy to transform the IS Architecture. (2023, January 6). GenMyModel. https://www.genmymodel.com/

GitUML. (n.d.). https://www.gituml.com/

Guindon, C. (n.d.-b). The Community for Open Innovation and Collaboration | The Eclipse Foundation. Eclipse Foundation. https://www.eclipse.org/

JetBrains: Developer Tools for Professionals and Teams. (n.d.). JetBrains. https://www.jetbrains.com/

NetBeans, A. (n.d.). Welcome to Apache NetBeans. https://netbeans.apache.org/

Rodina, D. (n.d.). Diagram CASE Tool for Software Modeling & Analysis - UML, BPMN, ERD. https://www.softwareideas.net/